Exit Ticket · reflect and retrieve

Exit Ticket

Binary search reliably finds a target in an unsorted array. True or false? Explain your answer and what low <= high protects.

Use key vocabulary from memory

1. Explain sorted array in your own words.

2. Explain target in your own words.

3. Explain midpoint in your own words.

Confidence and next step

How confident are you? Thumbs up / side / down

Which question would you improve after checking the classroom model?

Check answers after the lesson using the teacher-guided source models.

RHS COMPUTING / OCR H446 / LESSON 025 DRAFT · TEACHER REVIEW

Do Now · retrieval

Binary Search — From Trace Table to Working Code

Name: _____ Class: _____ Date: _____

Answer four retrieval questions before reviewing earlier learning.

Q1

What index identifies the first element of a C# array?

Q2

How does a linear search differ from a binary search?

Q3

Given int[] numbers = {3, 7, 12, 18, 24}; what is numbers[3]?

Q4

Why can binary search miss a target that exists in an unsorted array?

New Learning · key ideas

The important knowledge from today's lesson. Read, annotate, and explain it.

Core idea 1

Start with valid bounds — CODE:
low = 0;
high = numbers.Length - 1; — The last index is 7, not 8. Array length is not a valid index.

Core idea 2

Find the midpoint — CODE:
mid = low + (high - low) / 2;
numbers[3] = 18; — Integer division gives index 3. Compare its value (18) with target (31).

Core idea 3

Discard half — CODE:
18 < 31 → low = mid + 1;
low = 4; high = 7; — All positions through index 3 are too small. Keep only 4–7.

Annotate the key visual

Explain which labels or features show the main process.

You Do · extended practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

Question 3

Code and test in .NET Fiddle
Once your teacher has checked your whiteboard, type your function into .NET Fiddle or your school IDE. Call it from Main , run the program and check the returned position.
CODE:
BinarySearch(data, 35); // 5
BinarySearch(data, 4); // 0
BinarySearch(data, 57); // 7
BinarySearch(data, 20); // -1
Fix any errors independently. If time remains, explain why the search has time complexity O(log n).

Check that you have used the technical terms and explained your decisions.

You Do · independent practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

Question 1

Before you write
Discuss with your partner: What two inputs must BinarySearch receive? What integer must it return when the target is found or missing? Why must the array be sorted?
Then: finish your discussion, face forwards and work silently and independently.

Question 2

Write the complete function
Write a C# function named BinarySearch that accepts a sorted integer array and a target integer. Return the target's array index , or -1 if absent.
CODE:
int[] data = {4, 9, 13, 19, 26, 35, 42, 57};
int target = 35;
// Expected return value: 5
Write the function itself on your mini-whiteboard. Do not open .NET Fiddle yet.

New Learning · applying the ideas

Core idea 4

Stop when found or exhausted — CODE:
mid = 5; numbers[mid] = 31;
return 5; — If low becomes greater than high, return -1. Never assume the target exists.

Vocabulary · use the correct technical terms

sorted array — Values arranged in order, so comparisons can eliminate ranges.

target — The value the algorithm is trying to locate.

midpoint — The middle index of the current candidate range.

lower bound — The first index still under consideration.

Check your understanding

Explain one key process in your own words using two of the terms above.

More animated examples are available in the original online lesson.

I Do · worked example

Problem

Write a function that returns the target’s index

Follow the worked method

1. Declare an integer array — CODE:
int[] numbers = {3, 7, 12, 18, 24, 31, 45, 60}; — int = integer; [] = an array; numbers = its name; { } = the values; ; = end of the statement. The first index is 0.

2. Repeat and find the middle — CODE:
while (low <= high)
{
int mid = low + (high - low) / 2; — while repeats while a candidate remains; <= also checks when low and high are equal. Integer division gives a whole-number index.

3. Call the method and print the result — CODE:
int position = BinarySearch(numbers, target);
Console.WriteLine(position); // 5 — Call the function from Main after declaring the array and target. Its returned integer is stored in position and printed. Here 31 is at index 5; if the target is absent, position contains -1.

Why this method works

Identify the decision, rule or invariant that makes the method correct.

We Do · Try → Discuss → Improve

TRY independently in BLACK PEN; compare methods with a partner; improve your reasoning.

Guided problem · TRY

Understand the task
We have a sorted array. Search for the value 13. If it is present, return its index; otherwise return -1.
Which value are we searching for? What result should our function return if it is absent?

DISCUSS

Compare how you reached the answer. Where could someone make an error?

IMPROVE

Add one precise correction or a stronger reason. Answers are not printed here.